

Machine Learning Tek5 2026-2027 project

NICOLAS LE HIR
nicolas.lehir@epitech.eu

TABLE DES MATIÈRES

1	Introduction	1
1.1	Organization	1
1.1.1	Course material	1
1.2	Defense	1
1.2.1	Repository	2
1.2.2	Guidelines for the defense	2
2	Tasks	3
2.1	General ML knowledge (TODO)	3
2.2	Training time optimization on MNIST	3
2.3	Ultimate Vocal remover	4
2.4	Food 101	4
2.5	Tabular dataset (TODO)	4
2.6	Clustering on the TODO dataset	4

1 INTRODUCTION

In this project TODO

As this course is dedicated to discovering and exploring some of the many principles of machine learning, rather than being a production-oriented course, you are encouraged to explore original and personal approaches. It is not a huge deal if the final scores are not outstanding, as long as you took the chance to explore a custom approach and learned new methods.

1.1 Organization

1.1.1 Course material

You can find the course material at this address :

github.com/nlehir/ML_tek5/

1.2 Defense

There will be a final defense to present your work. Please make a couple of slides per task and respect the following maximum durations :

- 2.1 (4 min)
- 2.2 (7 min)

- 2.3 (7 min)
- 2.4 (7 min)
- 2.5 (7 min)
- 2.6 (7 min)

1.2.1 Repository

The project consists in several independent exercises. **Before the defense, please send me your repo url**, so that I can browse the code. You can push your work to the epitech repo dedicated to the project, inside the course team. Please do not send a fork of the course repo, with some added files. Instead, make a repo containing only your project files.

1.2.2 Guidelines for the defense

All processing should be made with python3. Please include only **only technical** information in the defense.

During the defense, what is expected is that you explain your process to arrive to the results. Unsuccessful trials and method are welcome! It is very important to draw **conclusions** for each task. For each task, please include some short docstrings (if using scripts) or markdown (if using notebooks) that make it easier to browse the code.

- **mandatory** : for each task (except for 2.1) :
 - define the objective with an exact ML vocabulary.
 - describe what the inputs and outputs of the model are. Recall the number of features and of samples, briefly describe the features.
- provide an **evaluation** or multiple evaluations of the obtained estimator(s), thanks to scorings of your choice.
- discuss the results obtained. To what extent have we solved the problem? Could the model be actually used? What precision can be expected with the model on unseen data?
- Example of explanations and discussions that are expected in the defense : discussion over the choice of models (model selection), hyperparameters, details on the optimization (or model training) process, and conclusions on which method(s) worked best, or over the choice of preprocessing methods on the datasets.
- Example of explanations **not** to include in the defense :
 - high level presentation of the course itself and of the tasks : please stay technical.
 - presentation of elementary python function, or library functions from torch, numpy, matplotlib, or the libraries themselves. You should **not** present these. You can however discuss specific functions or methods that are important for the optimization process, for instance in pytorch or tensorflow algorithms and optimizers (but again, you do not need to explain what pytorch is).
- be **quantitative** and precise about the results.
- when you mention the score of an estimator, **always** explicitly mention whether it is a train, test, validation, or cross-validation score (see the "scoring" chapter in the class)
- never forget to label the axes of plots
- never forget to mention the dimensions (unités) of the quantities
- if relevant or necessary, preprocess the data, and to justify this preprocessing. You could compare the estimator(s) obtained with and without preprocessing.

2 TASKS

2.1 General ML knowledge (TODO)

Define these concepts in your own words, and illustrate them with examples :

- overfitting
- feature, label
- gradient and gradient descent
- hyperparameter
- the difference between supervised and unsupervised learning

Why is gradient descent usually not performed on hyperparameters ?

2.2 Training time optimization on MNIST

As we have seen during the course, we can reach more than 0.99 of test accuracy in a couple of minutes on a laptop on the MNIST dataset (https://en.wikipedia.org/wiki/MNIST_database). However, in many machine learning applications, especially in embedded systems, there is a need to optimize the learning time and also the inference time.

Look for the **fastest possible way** to reach a 0.97 test accuracy, on a given hardware of your choice. The hardware that you use may be your laptop (you can also compare two laptops). You need to briefly present the hardware used (type of processor, frequency, number of cores, etc)

In this exercise, what we measure is the complete learning procedure : data loading + optimization time. We would like to **accelerate** it. Please also monitor the impact of the various methods on the **inference time**.

Super important : Please note that your objective is **not to simply reach the target test accuracy**. Your objective is to compare different methods to reach this accuracy as fast as possible. You can for instance isolate one parameter of the learning process, for example the learning rate, and plot the time to reach the target accuracy as a function of the learning rate. But this is just one possible example amongst many possibilities, don't hesitate to profile the learning and inference and to explore any acceleration method available.

Note that you can stop model training as soon as the target accuracy is reached.

It is required that you explore at least one method taken from a book or from a scientific article. You can find the book or the article yourself, or use one of the references presented during the course. For instance, part II of the Deep learning book (<https://www.deeplearningbook.org/>) contains many interesting discussions, but other sources are accepted as well. Any research is welcome, including research on libraries that are not as famous as pytorch and tensorflow, like jax.

Here are some suggestions :

- you can start from the architecture that we use in this example : https://github.com/nlehir/ML_tek5/tree/main/code/day_3/2_neural_networks/mnist and then try other methods to accelerate learning or inference time.
- try to **isolate** parameters by monitoring the change in speed after changing only one aspect of the pipeline, leaving the rest similar.

You are encouraged to present and analyse results of code profiling in order to find speed bottlenecks. Pay attention to the fact that profiling GPUs or pytorch code might have specificities.

2.3 Ultimate Vocal remover

<https://ultimatevocalremover.com/>

2.4 Food 101

<https://huggingface.co/datasets/ethz/food101>

2.5 Tabular dataset (TODO)

Perform a TODO on the TODO dataset

Provide a short, general analysis of the dataset, that studies its statistical properties, outliers, correlation matrices, or any other interesting analysis. You may produce visualizations. If you present statistical analysis, discuss whether it is consistent with intuition or not (instead of for instance just showing a correlation matrix).

compare several estimators / optimization procedures, from different points of view (scoring, training time, etc).

2.6 Clustering on the TODO dataset

Perform a clustering on the TODO dataset () Discuss the important algorithmic details and research : for instance, the **metric** and the algorithm(s) that were used. Again, it is very important to discuss the results obtained, and to interpret the clusters obtained. A numerical **evaluation** of the model is also expected. Note that several scores exist for clustering (<https://scikit-learn.org/stable/api/sklearn.metrics.html>). Use a scoring, or several scoring, that are meaningful with respect to the considered problem.